

SpaTE: Learning-Based Traffic Engineering under Sparse Traffic Observation in LEO Satellite Constellations

Linhui Wei, He Ma, Yumei Wang, *Member, IEEE*, and Guangteng Fan

Abstract—Low-Earth-orbit (LEO) satellite constellations require traffic engineering (TE) that allocates traffic within milliseconds over a network whose topology and demand shift continuously. Existing learning-based TE systems achieve this speed, but they assume a complete traffic matrix as input, whereas onboard resource limits and telemetry downlink costs mean a constellation controller can meter only a small fraction of demands at each decision epoch. Under such sparse observation, completing the matrix with simple zero or mean filling degrades allocation quality, and the feasibility-repair step that these systems inherit implicitly reads traffic information unavailable to a deployable controller. This paper presents SpaTE, a sparse-aware traffic engineering system that jointly designs the learned allocator and the repair layer around the observation actually available. SpaTE represents unobserved demands with a learnable mask embedding rather than fabricated values, propagates link state through a mask-aware gated graph neural network, encodes the history of sparse observations with a temporal Transformer, and drives repair with a neighbor-based demand estimate computed purely from the observation. On a 1,584-satellite Starlink constellation with population-driven traffic dynamics, SpaTE achieves lower maximum link utilization than zero-filling, mean-interpolation, and TEST-style baselines across observation ratios from 2% to 50% (**TODO: median improvement from overall_main.csv**), while retaining millisecond-level inference and transferring across constellation scales without retraining.

Index Terms—LEO satellite constellation, traffic engineering, sparse traffic measurement, maximum link utilization (MLU), graph neural networks.

1 INTRODUCTION

LOW-EARTH-ORBIT (LEO) satellite constellations are rapidly scaling toward tens of thousands of satellites [1], [2], [3], carrying user traffic with pronounced spatial and temporal fluctuations over inter-satellite links (ISLs) of limited capacity [4], [5]. Traffic engineering (TE), which centrally optimizes how each demand is split across candidate paths, has become a key control primitive for these networks [6], [7], [8], [9]. Classical solver-based TE recomputes allocations in minutes, far too slow for a topology and demand pattern that shift on the order of seconds, so a recent line of learning-based TE systems replaces the online solver with a neural allocator and cuts allocation time to milliseconds [10], [11], [12], [9], making centralized control of constellation-scale networks practical.

These systems, however, assume a *complete* traffic matrix as input, and this assumption is uniquely fragile in satellite networks. Obtaining a network-wide traffic matrix requires every satellite to meter, aggregate, and downlink fine-grained flow statistics through telemetry channels that compete with user traffic for scarce ISL and ground-station bandwidth, while consuming limited onboard computation and power; intermittent ground contacts further leave measurement gaps as the norm rather than the exception. A con-

stellation controller therefore observes a *sparse* traffic matrix, a fraction ρ of demands metered at each decision epoch, yet must still allocate *all* traffic, including the unmeasured part. Conversely, a TE system that acts well on sparse observations lets the operator deliberately meter only a small demand subset and reclaim the telemetry budget, which is exactly why sparse-observation TE is a natural operating mode for LEO constellations rather than a degraded one.

Sparse observation breaks existing learning-based TE in two places. First, the allocator: feeding a model trained for complete matrices with zero-filled input conflates “unmeasured” with “no traffic” and leaves no headroom on the links that unmeasured demands actually traverse, inflating the realized maximum link utilization (MLU), while two-stage complete-then-optimize pipelines propagate reconstruction errors into allocation decisions [13]. Second, and largely overlooked, the feasibility-repair step: because a neural allocator can emit allocations that overload individual links, learned TE systems append a few sweeps of a convex or heuristic repair procedure [10], [14], and the repair implementation inherited from the full-observation setting ranks congested links using the *ground-truth* traffic matrix, information that a sparsely measuring controller does not possess. Under sparse observation, the design question is therefore twofold: how should the learned allocator represent what it cannot see, and what information should the repair step be given so that it remains effective when fed only the observation.

This paper presents SpaTE (Sparse-aware Traffic Engineering), a learning-based TE system designed end

- L. Wei and G. Fan are with the National Innovation Institute of Defense Technology, Academy of Military Science, Beijing, China (e-mail: weilinhui998@163.com; fanguangteng@163.com).
- H. Ma and Y. Wang are with the School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: mahe@bupt.edu.cn; ymwang@bupt.edu.cn).
- Manuscript received XXX; revised XXX. (Corresponding author: Guangteng Fan.)

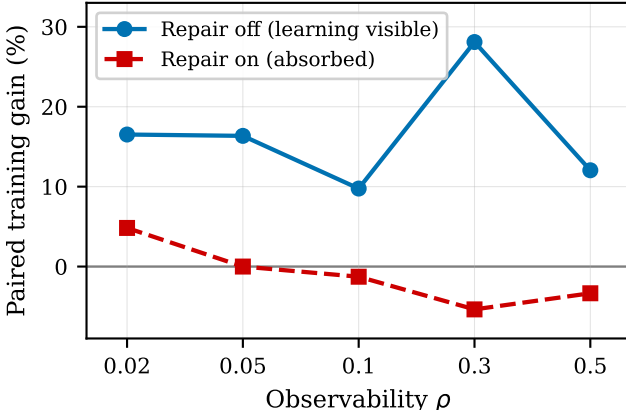


Fig. 1. Realized MLU versus observation ratio ρ on the 1,584-satellite constellation. SpaTE outperforms zero-filling, mean-interpolation, and TEST-style baselines at every ρ under the identical deployable repair layer. [TODO: regenerate from overall_main.csv]

to end around the observation actually available. On the allocator side, SpaTE represents each unobserved demand with a learnable mask embedding instead of a fabricated volume, propagates link state through a mask-aware gated graph neural network so that unobserved path nodes receive congestion information without injecting placeholder traffic, and encodes the history of sparse observations with a temporal Transformer fused into the graph representation by cross-attention. On the repair side, SpaTE restricts the rebalancing sweeps to a traffic estimate computed purely from the observation, filling each unobserved demand from observed demands that share its source. The two components add up: the learned allocator supplies a sparse-aware allocation prior that already avoids most overloads, and the observation-fed repair corrects the residual violations; the choice of the estimate that feeds repair, a neighbor-based fill rather than zeros, is what keeps the repair step effective at low observability.

We evaluate SpaTE on a 1,584-satellite Starlink Shell-1 constellation with population-driven traffic dynamics, sweeping the observation ratio ρ from 0.02 to 0.5 and comparing against zero-filling and mean-interpolation adaptations of full-observation systems and a TEST-style sparse baseline [13], all methods paired with the identical deployable repair layer. SpaTE achieves the lowest realized MLU at every observation ratio tested ([TODO: headline numbers from overall_main.csv, e.g., median MLU reduction over the best baseline per ρ]), retains millisecond-level inference on the full constellation, and transfers to a one-third-scale constellation instance without retraining (Fig. 1).

In summary, this paper makes the following contributions:

- We formulate the sparse-observation TE problem for LEO constellations, in which the controller meters only a fraction ρ of demands per epoch yet is evaluated by the realized MLU that all traffic, observed or not, induces on the network.
- We design SpaTE, a sparse-aware learned allocator that combines learnable mask embeddings, mask-aware gated message passing, and temporal cross-

attention, so that missing measurements are represented explicitly rather than filled with fabricated volumes.

- We identify that the feasibility-repair step inherited from full-observation TE reads the ground-truth traffic matrix, and we design a deployable alternative whose rebalancing is driven by a neighbor-based estimate computed from the observation alone, showing how the information source handed to repair determines its effectiveness under sparsity.
- We evaluate SpaTE at Starlink scale across observation ratios from 2% to 50%, demonstrating consistently lower realized MLU than sparse adaptations of existing systems, millisecond-level allocation latency, and retraining-free transfer across constellation scales.

The remainder of this paper is organized as follows. Section 2 reviews related work on TE for satellite networks and learning-based TE. Section 3 formulates the sparse-observation TE problem. Section 4 presents the design of SpaTE. Section 5 reports the experimental evaluation, and Section 6 concludes the paper.

2 RELATED WORK

2.1 ML-Driven Traffic Engineering for WANs

Traffic engineering has long been formulated as a constrained optimization problem [15], [16] solved by linear programming (LP), whose runtime scales poorly with network size. SMORE [17] decouples robust oblivious path selection from online rate adaptation, while NCFlow [18] and POP [19] accelerate LP by problem decomposition, trading allocation quality for speed. A recent line of work replaces online solvers with learned models [20]: DOTE [11] directly maps historical traffic matrices to split ratios with stochastic optimization; Teal [10] combines a flow-centric GNN, multi-agent RL, and ADMM fine-tuning to achieve near-optimal allocations with orders-of-magnitude speedups; FIGRET [21] enhances robustness against traffic uncertainty at per-flow granularity. To survive topology changes without retraining, HARP [12] enforces invariances to node permutation and tunnel reordering, transferring across topologies unseen in training; Aether [22] pursues the same generality with elastic multi-agent graph transformers, and path-based GNNs have been trained for robust distributed TE under link failures [23]. More recently, large language models have been explored as general-purpose traffic planners [24]. However, all of these systems assume the *complete* traffic matrix is available at decision time, an assumption that rarely holds in large-scale satellite networks, where network-wide measurement is prohibitively expensive.

2.2 TE and Routing for Satellite Networks

LEO mega-constellations pose unique challenges for TE: topologies change every few tens of milliseconds as satellites move, and control decisions must be produced within stringent latency budgets [1], [2], [4]. Classical approaches absorb mobility through topology abstractions: the virtual node model binds logical nodes to earth-fixed footprints so that upper-layer routing observes a quasi-static grid [25].

Optimization-based satellite TE has explored segment routing [26], distributed clustering that optimizes elephant flows separately [27], and supervised learning for distributed split-ratio prediction [28]. SaTE [9] is the state-of-the-art learning-based TE for satellite constellations: it formulates a heterogeneous graph over satellites, demands, paths, and links, prunes zero-traffic relations to make training tractable on a single GPU, and selects representative topologies via Graph2Vec and DPP sampling so that one trained model generalizes across topology snapshots with millisecond inference. In parallel, FNC [29], [30] targets dynamic traffic allocation for earth-observation constellations, replacing RL with end-to-end imitation learning and substituting iterative constraint solvers with normalization-based feasibility layers. Notably, FNC's imitation learning requires expert demonstrations, and the TE expert is an optimal allocation whose computation needs the complete demand matrix of each episode, a paradigm that fundamentally requires complete post-hoc observability, which is unavailable under sparse measurement. Both SaTE and FNC, like their WAN counterparts, assume the traffic matrix is fully observable at decision time.

2.3 TE with Incomplete Traffic Observations

Network-wide traffic measurement is expensive: collecting a complete traffic matrix requires instrumenting all ingress nodes at fine time granularity, which is particularly onerous on resource-constrained satellite platforms. A classical workaround is a two-stage pipeline: first estimate or complete the traffic matrix via network tomography [31], [32], matrix completion, or compressive sensing [33], then optimize routing on the estimate. However, reconstruction errors propagate to and degrade the downstream allocation. TEST [13] is, to our knowledge, the first end-to-end approach that learns routing policies directly from sparsely measured traffic matrices, integrating a Transformer over historical sparse sequences with a GCN over the topology; it fills unobserved entries with zeros, compensates with synthesized training augmentation, and targets terrestrial networks with static topologies. In the graph learning literature, message passing under missing node features has been addressed via teacher-student distillation [34], but such techniques have not been applied to network TE. No prior work, to our knowledge, performs TE from sparse traffic observations in satellite networks, where the difficulty is compounded by time-varying topologies and drifting demand sets.

2.4 Positioning

Our work sits at the intersection of these three lines (Table 1). Unlike SaTE and FNC, which assume fully observable demands, SpaTE performs TE from sparse observations, down to a 2% observation ratio. Unlike TEST, which fills unobserved entries with zeros and targets static terrestrial networks, SpaTE replaces missing entries with learnable mask embeddings, gates their message passing, and handles LEO dynamics, namely time-varying topologies and drifting demand-pair sets, via demand pruning with size-invariant model weights, enabling train-once, zero-retraining deployment. Finally, unlike all of the above, SpaTE also redesigns

TABLE 1
Comparison with the related works.

	Teal [10]	SaTE [9]	TEST [13]	FNC [29]	Ours
Satellite scenario	×	✓	×	✓	✓
Dynamic topology	×	✓	×	✓	✓
Sparse observation	×	×	✓	×	✓
Temporal modeling	×	×	✓	×	✓
Mask-aware model	×	×	×	×	✓
Observation-fed repair	×	×	×	×	✓
Demand-set drift	×	partial	×	×	✓

TABLE 2
Main Notations.

Symbol	Definition
$G^t = (V, E^t)$	constellation graph at epoch t ; c_e link capacity
D^t, d_i	demand matrix; volume of demand i
$\mathcal{K}^t$	active demand set at epoch t
κ, P_i	candidate paths per demand; path set of demand i
$r_i, f_{i,k}$	split ratios of demand i ; flow on path $p_{i,k}$
M, ρ	binary observation mask; observability ratio
O^t, H^t	sparse observation $D^t \odot M$; last- L history
ϕ	learnable mask embedding for unmeasured demands
$A, \tilde{A}$	bipartite adjacency; its mask-gated variant
H_S, H_T, H_F	spatial, temporal, and fused embeddings
π_θ, R	TE policy network; ground-truth reward

the feasibility-repair step so that its rebalancing reads only an estimate computed from the observation, rather than the ground-truth traffic matrix that full-observation systems implicitly consult.

3 MODEL AND PROBLEM FORMULATION

3.1 Network and Traffic Model

We model the constellation at control epoch t as a directed graph $G^t = (V, E^t)$, where V is the set of N satellites and E^t the set of inter-satellite links (ISLs), each link e with capacity c_e . Time is slotted into control epochs; within an epoch the topology snapshot is fixed, while across epochs both E^t and the traffic change as satellites move.

Traffic is described by a demand matrix $D^t \in \mathbb{R}^{N \times N}$, where d_i denotes the volume of the i -th source-destination pair. Since satellite traffic is geographically concentrated, we maintain an *active demand set* $\mathcal{K}^t$, the pairs with nonzero demand in recent history, and only these are modeled. Each demand $i \in \mathcal{K}^t$ is served by κ pre-selected candidate paths $P_i = \{p_{i,1}, \dots, p_{i,\kappa}\}$ ($\kappa = 4$ edge-disjoint shortest paths in our implementation). The TE decision for demand i is a split-ratio vector $r_i \in \mathbb{R}^\kappa$ with $r_{i,k} \geq 0$ and $\sum_k r_{i,k} \leq 1$, allocating flow $f_{i,k} = r_{i,k} \cdot d_i$ to path $p_{i,k}$; the residual $1 - \sum_k r_{i,k}$ represents intentionally unserved traffic. Main notations are summarized in Table 2.

3.2 TE with Complete Observation

With the complete demand matrix available, TE is the classic path-based allocation problem, for which two objectives dominate the literature. *Load balancing* minimizes the maximum link utilization (MLU), the standard objective in

satellite TE, where hot ISLs over populated regions dictate quality of service [9], [26], [27]:

$$\min_{\{r_{i,k}\}} \theta \quad (1)$$

$$\text{s.t.} \quad \frac{1}{c_{e(i,k):e \in p_{i,k}}} \sum r_{i,k} d_i \leq \theta, \quad \forall e \in E^t, \quad (2)$$

$$\sum_{k=1}^{\kappa} r_{i,k} = 1, \quad \forall i \in \mathcal{K}^t, \quad (3)$$

$$r_{i,k} \geq 0, \quad \forall i, k, \quad (4)$$

where Eq. (3) enforces flow conservation: every demand must be fully served, so θ cannot be lowered by simply withholding traffic. *Throughput maximization* instead maximizes the total satisfied demand, permitting demands to be served only partially when capacity is scarce (total-flow TE [10], [18]):

$$\max_{\{f_{i,k}\}} \sum_{i \in \mathcal{K}^t} \sum_{k=1}^{\kappa} f_{i,k} \quad (5)$$

$$\text{s.t.} \quad \sum_{k=1}^{\kappa} f_{i,k} \leq d_i, \quad \forall i \in \mathcal{K}^t, \quad (6)$$

$$\sum_{(i,k): e \in p_{i,k}} f_{i,k} \leq c_e, \quad \forall e \in E^t, \quad (7)$$

$$f_{i,k} \geq 0, \quad \forall i, k. \quad (8)$$

Both problems are linear programs solvable to optimality, but at constellation scale their solve time far exceeds the control epoch [9]. More fundamentally for this paper, the link-load terms in Eqs. (2) and (7), as well as the per-demand cap in Eq. (6), all require knowing d_i for *every* active demand. We treat MLU minimization as the primary objective throughout this paper, following the satellite TE literature, and report throughput maximization as a secondary instantiation to show that our design is objective-agnostic.

3.3 TE with Sparse Observation

3.3.1 Observation Model

At each epoch only a subset of demands is measured. A binary mask $M \in \{0, 1\}^{|\mathcal{K}^t|}$ marks demand i as measured ($M_i = 1$) or not ($M_i = 0$), and the controller receives the sparse observation

$$O^t = D^t \odot M, \quad \text{together with } M \text{ itself}, \quad (9)$$

where $\odot$ is the element-wise product. Fig. 2 contrasts a ground-truth traffic matrix from our Starlink trace with its node-level sparse observation. We consider two measurement granularities: *flow-level*, where a fraction ρ of demand pairs is sampled, and *node-level*, where a fraction ρ of satellites meter all their outgoing demands; the latter matches deployments where measurement capability resides on a subset of satellites. The controller additionally retains the last L sparse observations $H^t = (O^{t-L+1}, \dots, O^t)$.

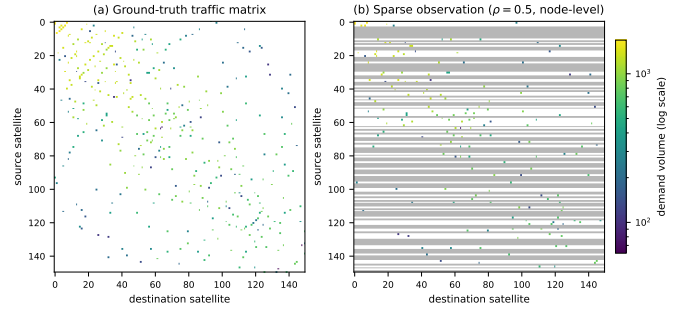


Fig. 2. A ground-truth traffic matrix from the Starlink 22×72 trace (150 busiest satellites shown) and its node-level sparse observation: white cells carry no traffic, gray rows are unmeasured source satellites, and colored cells are observed demands.

3.3.2 Problem Statement

A key observation makes decision-making under unknown demands physically executable: the decision variables are split *ratios*, not absolute volumes. Once installed in the forwarding plane, r_i is applied multiplicatively to whatever traffic actually arrives, so a decision for demand i requires no knowledge of d_i . Sparse-observation TE then seeks a policy π that maps the observable state to split ratios for *all* active demands,

$$\{r_i\}_{i \in \mathcal{K}^t} = \pi(H^t, M, G^t), \quad (10)$$

maximizing the *ground-truth* objective. Let $f_{i,k}(\pi) = \tilde{r}_{i,k} \cdot d_i$ denote the flow actually delivered on path $p_{i,k}$, where $\{\tilde{r}_i\}$ are the policy's ratios after an optional repair step: ADMM refinement with rounding under the throughput objective, and conservation-preserving rebalancing under MLU, where the ratios are already conservation-feasible by Eq. (3) but may still overload individual links. Formally, for MLU minimization

$$\min_{\pi} \mathbb{E}_t \left[\max_{e \in E^t} \frac{1}{c_e} \sum_{(i,k): e \in p_{i,k}} f_{i,k}(\pi) \right], \quad (11)$$

and, for the throughput variant,

$$\max_{\pi} \mathbb{E}_t \left[\sum_{i \in \mathcal{K}^t} \sum_{k=1}^{\kappa} f_{i,k}(\pi) \right], \quad (12)$$

where the expectations are over traffic and topology dynamics. Note that the link loads in Eq. (11) are evaluated with the true demands d_i : an unmeasured demand still consumes real capacity even though the policy could not see it, so misjudging it inflates the realized MLU. The problem is a partially observable sequential decision problem: unlike Eqs. (1) to (8), where the same D^t appears in both the decision and the constraints, here the decision variables and the evaluation live in different information sets: the policy acts on (O, M) only, while feasibility and performance are settled by the hidden D . Intuitively, the policy is graded against an exam it never gets to read in full.

3.4 Solution Paradigm

Three solution families exist for Eqs. (11) and (12). *Solve-with-zeros* plugs O directly into the LP in Eqs. (1) to (8), implicitly assuming unmeasured demands are zero; the resulting policy leaves no headroom on the links that unmeasured

traffic actually traverses, so the realized MLU can be far above the optimum (and, under the throughput objective, unmeasured demands are structurally starved). *Two-stage* approaches first estimate $\hat{D}$ from (H, M) , via interpolation, tensor completion, or compressive sensing [31], [32], [33], and then solve the LP on $\hat{D}$; estimation is optimized for reconstruction error, which is misaligned with allocation quality, and errors propagate un-damped into the allocation. We instead adopt *end-to-end learning*: parameterize π_θ as a neural network and optimize θ directly against the TE objective,

$$\max_{\theta} \mathbb{E}[R(\pi_\theta(H, M, G), D)], \quad (13)$$

where R is the TE objective evaluated on the true D : negative realized MLU for Eq. (11), or total satisfied demand for Eq. (12). Since R is non-differentiable through the capacity-constrained network and the true D is available only as a training-time signal, we optimize Eq. (13) with multi-agent reinforcement learning, in which each demand acts as an agent sharing θ and receives a COMA-style marginal-contribution reward, followed by an optional lightweight repair step at inference that relieves overloaded links. This paradigm makes the observation model in Eq. (9) a property of the *input*, and the optimization target in Eq. (12) a property of the *training signal*, cleanly separating what the policy can see from what it is optimized for.

4 SPATE DESIGN

4.1 Overview

SpaTE solves the sparse-observation TE problem of Section 3.3 in four stages, as illustrated in Fig. 3. Given the sparse observation (O^t, M) and history H^t , (i) unmeasured demands are represented by a learnable mask embedding (Section 4.2); (ii) a mask-aware gated GNN propagates information between candidate paths and links without letting placeholders contaminate link states (Section 4.3); (iii) a Transformer encodes the observation history and fuses it with spatial embeddings via per-demand cross-attention (Section 4.4); (iv) a shared policy head outputs split ratios per demand, refined by the constraint-repair step (Section 4.6). All learnable weights are shared across demands and independent of the demand-set size, which Section 4.5 exploits for zero-retraining deployment under constellation dynamics. Algorithm 1 summarizes one inference epoch.

4.2 Mask-Aware Demand Representation

A sparse observation is fundamentally ambiguous: a zero entry may mean “no traffic” or “not measured”. Zero-filling, the de-facto treatment in prior sparse TE [13], collapses the two cases and systematically biases the allocator toward starving unmeasured demands. SpaTE instead assigns each unmeasured demand a *learnable mask embedding* $\phi \in \mathbb{R}^\kappa$ (one scalar per candidate path):

$$x_i = M_i \cdot d_i + (1 - M_i) \cdot \phi, \quad (14)$$

so the model learns a data-driven prior for missing traffic during end-to-end training, and, critically, the input itself distinguishes “not measured” from “zero”. Learnable imputation has proven effective for incomplete graph features [34]; we bring it to TE, where the imputed value

Algorithm 1 SpaTE inference at control epoch t

Require: sparse history H^t , mask M , topology G^t , weights θ^*

Ensure: split ratios $\{r_i\}_{i \in \mathcal{K}^t}$

- 1: $\mathcal{K}^t \leftarrow$ active demand set from recent history $\triangleright$ pruning, Section 4.5
- 2: rebuild bipartite graph and gated adjacency $\tilde{A}$ from $\mathcal{K}^t, M$ $\triangleright$ Eq. (15)
- 3: $x_i \leftarrow M_i d_i + (1 - M_i) \phi$ for all i $\triangleright$ mask embedding, Eq. (14)
- 4: $H_S \leftarrow \text{GATEDFLOWGNN}(x, \tilde{A})$ $\triangleright$ spatial embeddings
- 5: $H_T \leftarrow \text{TEMPORALENCODER}(H^t)$ $\triangleright$ temporal embeddings
- 6: $H_F \leftarrow \text{CROSSATTENTION}(H_T, H_S)$ $\triangleright$ Eq. (16)
- 7: $\{r_i\} \leftarrow \text{POLICYHEAD}([H_S \parallel H_F])$
- 8: $\{r_i\} \leftarrow \text{REPAIR}(\{r_i\}, G^t, \hat{D}^t)$ $\triangleright$ Section 4.6
- 9: **return** $\{r_i\}$

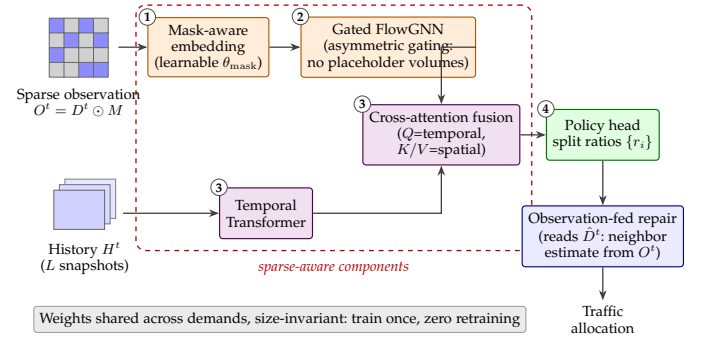


Fig. 3. SpaTE overview: a sparse observation is embedded with mask awareness, processed by a gated flow-centric GNN in parallel with a temporal encoder, fused via per-demand cross-attention, and mapped to split ratios refined by the observation-fed repair step, whose traffic estimate is computed from the same sparse observation.

directly steers resource allocation. The embedding is trained jointly with the policy under the TE objective in Eq. (13) rather than a reconstruction loss, so ϕ converges to values that maximize allocation quality, not reconstruction accuracy.

4.3 Gated Message Passing

SpaTE inherits Teal’s flow-centric graph [10]: candidate paths and physical links form a bipartite structure where a PathNode connects to the EdgeNodes it traverses, and message passing alternates between them with GCN-style normalized aggregation [35]. Under sparse observation, however, symmetric message passing is harmful: placeholder values on unmeasured PathNodes would propagate into EdgeNode embeddings and corrupt the network-wide estimate of link utilization, a phenomenon consistent with the feature-structure interference observed in incomplete graphs [34]. We therefore gate the propagation *asymmetrically*. Let A be the normalized bipartite adjacency; we zero out entries that carry path-to-edge messages from unmeasured paths while keeping all edge-to-path entries:

$$\tilde{A}[e, p] = M_p \cdot A[e, p], \quad \tilde{A}[p, e] = A[p, e], \quad (15)$$

where M_p is the demand-level mask lifted to path p . Unmeasured demands thus remain *listeners*: they sense link states, which are shaped by measured traffic, but inject no fabricated volume into them. The gate is a fixed binary function of the mask with no extra parameters, adding zero inference overhead. The asymmetric gate yields a formal isolation property:

Proposition 1 (Placeholder isolation). Under the gated propagation in Eq. (15), the embeddings of all EdgeNodes and of all PathNodes belonging to measured demands are independent of the mask embedding ϕ ; ϕ influences only the allocations of unmeasured demands.

Proof: Consider first the spatial stream, by induction on GNN layers. At layer 0, only unmeasured PathNodes carry ϕ . At layer l , an EdgeNode aggregates path-to-edge messages with weights $\bar{A}[e, p] = M_p A[e, p]$, which vanish for every unmeasured path; hence EdgeNode embeddings depend only on measured-path inputs and their own history, none of which contains ϕ . A measured PathNode aggregates EdgeNode embeddings (ϕ -free by the above) and its own ϕ -free input; the per-demand layers mix only paths of the same demand, which share the same mask value. The temporal and fusion streams preserve the property: the temporal encoder acts per path on the raw observation history H^t , which contains no ϕ by construction, and the cross-attention of Eq. (16) operates strictly within each demand, so the fused features of a measured demand combine only its own ϕ -free temporal and spatial embeddings. Thus ϕ never enters any ϕ -free branch, and only unmeasured PathNodes, and consequently their own split ratios, depend on ϕ . $\square$

In other words, learning a poor prior ϕ can at worst misallocate the unmeasured demands themselves, but can never corrupt the allocations of measured traffic, a robustness guarantee that zero-filling with ungated propagation does not provide.

4.4 Temporal-Spatial Fusion

Satellite traffic exhibits strong short-term temporal correlation; the recent history of sparse observations therefore carries recoverable information about currently missing entries. SpaTE encodes, for each candidate path, its observation sequence over the last L epochs with a lightweight Transformer [36]: scalar volumes are log-compressed, which we find essential for training stability, and projected to d_{model} dimensions, summed with learnable positional embeddings, and passed through a Transformer encoder; the representation at the latest step forms the temporal embedding H_T . Rather than concatenating temporal and spatial features naively, we fuse them with *per-demand cross-attention*: within each demand, temporal embeddings of its κ candidate paths act as queries attending to the κ spatial (GNN) embeddings H_S as keys and values,

$$H_F = \text{softmax}\left(\frac{W_q H_T (W_k H_S)^\top}{\sqrt{d}}\right) W_v H_S, \quad (16)$$

and the fused features $[H_S \parallel H_F]$ feed the policy head. Intuitively, the temporal stream decides *which* spatial evidence to trust for each path: when the current entry is missing,

attention shifts toward paths and links whose states corroborate the historical pattern. The full pipeline infers in under 0.4 s per snapshot on the 1,584-satellite constellation, well within the 5-minute control-loop period.

4.5 Scaling to Dynamic Constellations

4.5.1 Demand Pruning

Modeling all $N(N - 1)$ pairs of a 1,584-satellite constellation yields 2.5M demands and tens of millions of path variables, which is infeasible for online control. Satellite traffic, however, is geographically concentrated: only 0.25% of pairs ever carry traffic in our dataset. SpaTE instantiates PathNodes only for the demand-pair union observed in historical traffic matrices, shrinking the graph by $\sim 400\times$ with no loss of served traffic, in the same spirit as SaTE's traffic pruning [9].

4.5.2 Zero-Retraining under Drift

Pruned demand sets are not static: as satellites move, the active pair set drifts (we measure $\sim 10\%$ turnover across our trace). Rather than retraining, SpaTE exploits a structural property of its architecture: every learnable component (GNN layers, mask embedding, temporal encoder, fusion and policy heads) is shared across demands and independent of the demand-set size. The demand set, candidate paths, and the bipartite graph are runtime *inputs*, not parameters. At each control epoch the controller rebuilds the graph from the current active set (a millisecond-scale index operation on +Grid constellations, where candidate paths depend only on the orbital displacement between endpoints and reduce to $O(N)$ path templates) and reuses the trained weights as-is. Demands outside the modeled set, necessarily sporadic mice flows, fall back to shortest-path routing until the next set refresh. This reuse is justified by the following structural property:

Proposition 2 (Size invariance and permutation equivariance). The parameter set of SpaTE is independent of $|\mathcal{K}^t|$, N , and $|E^t|$, and the mapping from inputs to split ratios is equivariant to any permutation of the demand indices.

Proof: Every learnable component is a per-node or per-demand operator with shared weights: the mask embedding ϕ and temporal encoder act on each path token, the GNN layers combine node-wise linear maps with sparse aggregation, the fusion and policy heads act within each demand. None of their parameter shapes involves $|\mathcal{K}^t|$, N , or $|E^t|$. Permuting demand indices permutes the rows of every input and of the bipartite adjacency consistently; since shared node-wise operators commute with permutations and sparse aggregation is index-based, all intermediate embeddings, and hence the outputs, are permuted identically. $\square$

4.6 Training and Constraint Repair

SpaTE trains with the multi-agent RL scheme of Teal [10]: each demand is an agent sharing the policy network. The policy outputs a Gaussian mean per path, and actions are sampled during training. To assign credit, each agent

receives a COMA-style [37] counterfactual reward that estimates its marginal contribution to the global objective in Eq. (13): for demand i ,

$$A_i = \mathbb{E}_{\tilde{r}_i \sim \pi'} [R(r_{-i}, r_i) - R(r_{-i}, \tilde{r}_i)], \quad (17)$$

where r_{-i} denotes the allocations of all other demands and $\tilde{r}_i$ is a counterfactual action resampled from a reference distribution π' ; the policy gradient weights each agent's log probability by A_i . Algorithm 2 summarizes offline training. Two points deserve emphasis. First, the reward is computed against *ground-truth* demands during offline training (sparse observation constrains the policy input, never the training signal), which is exactly what a simulator or historical trace provides. Second, we deliberately retain reward-driven RL rather than imitation learning [29]: imitation learning requires expert allocations solved from *fully realized* demands of each episode, a supervision signal that is unavailable when even post-hoc observation is sparse; RL needs only the reward, which the network itself provides.

Constraint repair. Under the throughput objective, allocations are refined by two parallelizable ADMM [14] iterations and a rounding pass that together repair residual violations of the capacity constraint in Eq. (7). Under the MLU objective, our primary setting, the situation differs. The softmax ratios already satisfy flow conservation in Eq. (3) by construction, so there is no infeasibility to project away; what they can still do is overload individual links. The repair step is therefore not a projection but a *conservation-preserving rebalancing*: for each demand it repeatedly shifts a fraction of the flow from its most congested candidate path, ranked by the bottleneck utilization of the links that path traverses, onto its least congested one, leaving per-demand totals untouched so that MLU cannot be lowered by underserving traffic. One unit of repair budget corresponds to ten such sweeps.

Observation-fed repair. Ranking paths by bottleneck utilization requires link loads, and link loads require demand volumes. In the fully observed WAN setting this repair mechanism was designed for, those volumes are available at decision time, so consulting them is unremarkable. Under sparse observation they are not: a controller that meters only a fraction ρ of demands cannot compute the true utilization, so the inherited implementation, which reads the ground-truth matrix to rank paths (denoted ORACLE), is not deployable. SpaTE instead drives repair with a traffic estimate $\hat{D}^t$ computed purely from the observation. The estimate itself is a design choice, and it matters. Zero-filling the unobserved entries (OBS-ZERO) makes repair blind to the load that unmeasured traffic places on links, so it misranks congested paths exactly where repair is needed most. SpaTE therefore fills each unobserved demand with a *neighbor estimate*: the volume inferred from observed demands sharing the same source satellite, scaled to preserve the observed per-source totals (OBS-NBR). This exploits the geographic concentration of satellite traffic, in which demands from the same satellite footprint are strongly correlated, and it gives the rebalancing sweeps a utilization estimate that ranks paths nearly as well as the ground truth. Choosing what repair reads is thus the second half of sparse-aware design: the learned allocator supplies a sparse-aware allocation prior, and the observation-fed repair corrects residual overloads,

Algorithm 2 Offline training of SpaTE

Require: historical trace $\{(D^t, G^t)\}$, mask M , learning rate η

Ensure: trained weights θ^* (incl. mask embedding ϕ)

- 1: build demand set $\mathcal{K}$ and graph from training-slice TMs
- 2: **for** each epoch **do**
- 3: **for** each snapshot t in the training slice **do**
- 4: form sparse input (H^t, M) ; sample actions $r \sim \pi_\theta(\cdot \mid H^t, M, G^t)$
- 5: compute counterfactual rewards $\{A_i\}$ via Eq. (17)
- 6: **using the true** D^t
- 7: $\theta \leftarrow \theta + \eta \sum_i A_i \nabla_\theta \log \pi_\theta(r_i)$
- 8: **end for**
- 9: **return** θ^*

with the neighbor estimate keeping the correction effective at low observability. We quantify the gap among the three information sources in the evaluation.

4.7 Complexity Analysis

Let $D = |\mathcal{K}^t|$ be the number of active demands, κ the candidate paths per demand, E the number of ISLs, and $\bar{\ell}$ the average path length. The gated FlowGNN performs message passing over the bipartite graph whose edge count is $O(D\kappa\bar{\ell})$; with Λ layers its cost is $O(\Lambda D\kappa\bar{\ell}d)$ for hidden width d . The temporal encoder applies a Transformer of length L independently to each of the $D\kappa$ path tokens, costing $O(D\kappa L^2 d)$, and the per-demand cross-attention over κ paths adds $O(D\kappa^2 d)$. Since κ , L , $\bar{\ell}$, and d are small constants fixed by design, the end-to-end inference cost is *linear* in the number of active demands, $O(D)$, and all operations are batched across demands on the GPU. Crucially, the parameter count is independent of D , N , and E : the same weights apply to any demand set, which is what enables zero-retraining reuse (Section 4.5). In our configuration the entire model has merely 4,468 trainable parameters, small enough to be replicated on resource-constrained satellite platforms. Demand pruning reduces D from $N(N-1)$ to the observed support (roughly $400\times$ on our constellation), turning an otherwise intractable problem into one solved well within the control period.

5 EVALUATION

5.1 Setup

Constellation. Our main testbed is a Starlink Shell-1 Walker constellation [3] interconnected in a +Grid pattern (Table 3); orbital dynamics are derived from the Satellite Tool Kit (STK). With 1,584 satellites, an average path length of 23.51 hops, and a diameter of 47, capacity conflicts between demands are far more frequent than in the terrestrial WANS used in prior TE studies [10]. For the cross-scale check (Section 5.6) we use a one-third-scale instance of the same architecture (24 planes $\times$ 22 satellites, 528 nodes).

Traffic. Traffic matrices are generated by random city sampling [38]: cities are drawn with probability proportional to their population, each sampled city pair is attached to its nearest satellites, and per-pair flows are aggregated

TABLE 3
Simulation parameters of the Starlink constellation.

Parameter	Value
Height of LEO satellites	550 km
Constellation configuration	Walker
Total number of satellites	1,584
Number of orbital planes	72
Satellites per plane	22
Inclination of orbits	55°
Phase factor	3
Inter-satellite links	6,336

TABLE 4
Statistics of the population-based traffic trace (Mbps).

Metric	Value
Snapshots (5-min interval)	101
Nonzero demand, max	1,800
Nonzero demand, mean±std	159.5 ± 242.3
Total demand per snapshot	890,242
Active pairs per snapshot	5,583
Active-pair union (trace)	6,334

into satellite-level demands (Table 4). We use 101 consecutive snapshots (one every 5 minutes), with snapshots 0–79 for training, 80–89 for validation, and 90–100 for testing, a strict temporal split with no leakage. ISL capacity is calibrated so that the fully-observed optimum leaves moderate congestion, keeping TE decisions non-trivial. Demand pruning (Section 4.5) restricts the modeled demand set to the ~6.3K pairs that ever carry traffic, out of 2.5M node pairs.

Metric and protocol. Our metric is the *realized MLU* evaluated against ground-truth demands (lower is better); flow conservation in Eq. (3) is enforced by construction, so MLU cannot be gamed by under-serving traffic. We sweep the observation ratio $\rho \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$ and run three random seeds per configuration. Unless stated otherwise, every method is paired with the *identical deployable repair layer*: two rebalancing sweeps whose utilization estimate reads only the sparse observation with neighbor fill (OBS-NBR, Section 4.6), so that no method receives information a sparsely measuring controller would not have, and the only factor that varies across methods is the allocator. The ORACLE repair variant, which reads the ground-truth matrix as inherited implementations do, appears only in the analysis of Section 5.4. Because seed-to-seed variation is comparable to between-configuration differences, headline comparisons are *paired* on (ρ, seed) and assessed with two-sided sign tests; we report win counts and medians rather than pooled means.

Baselines. All learned baselines share the training pipeline, path sets, and repair layer of SpaTE and differ only in how they handle sparsity:

- *zero-fill*: the sparse adaptation of full-observation systems such as Teal [10] and SaTE [9], which cannot run on an incomplete matrix; unobserved entries are set to zero and the model is trained end to end on the filled input. SaTE’s heterogeneous-graph allocator targets the same MLU objective with the

TABLE 5
Model and training hyperparameters (defaults held fixed across all ρ and seeds).

Parameter	Value
FlowGNN layers Λ	6
Candidate paths κ	4
Temporal d_{model} / heads / layers	16 / 2 / 1
Fusion dim d_{fuse}	8
History length L	3
COMA samples per update	30
Mask-embedding init	57.0 (median nonzero demand)
Learning rate	1×10^{-4}
Epochs (warmup / early stop)	60 (4 / on)
Restart screening	3 inits
Batch size	20
Untrained control	0 epochs, 1 init
Repair budget (deployed)	2 sweeps-of-ten
Trainable parameters	4,468

same pruning, so this variant is its closest sparse-input proxy.

- *mean-interp*: a two-stage complete-then-optimize baseline that fills unobserved entries with the mean of observed demands before allocation.
- *nbr-fill*: a stronger two-stage baseline that fills each unobserved demand from observed demands sharing its source satellite, the same estimate SpaTE hands to repair.
- *TEST-style*: a faithful re-implementation of TEST [13] in our pipeline: zero-filled input, a Transformer over the observation history ($L = 3$), and an ungated GCN.
- *untrained*: the SpaTE architecture with weights frozen at random initialization, isolating the contribution of training.
- *full observation*: SpaTE given the complete matrix ($\rho = 1$), an upper reference on input information.
- *LP oracle*: the per-snapshot optimum of Eqs. (1) to (4) solved on ground-truth demands, a lower bound no online method can beat.

Implementation. Table 5 lists the hyperparameters, drawn from the code defaults and held fixed across all ρ and seeds. The untrained control shares the trained model’s random initialization and differs only in skipping gradient updates (epochs=0); it is therefore a clean ablation of the *training* factor with identical architecture, mask embedding, and repair layer.

5.2 Overall Performance

Table 6 reports the median realized MLU of SpaTE and all baselines across the full observability range under the deployable protocol, and Fig. 1 plots the corresponding curves. [TODO: Fill Table 6 from overall_main.csv (3 seeds per cell) once the server batch completes; update the win counts and percentages below accordingly.]

Three observations stand out. First, SpaTE attains the lowest median MLU at [TODO: k of 5] observation ratios, with a median improvement of [TODO: XX%] over the strongest baseline per column; the advantage is largest at low ρ , where the input carries the least information and the

TABLE 6

Overall comparison: median realized MLU (3 seeds) under the deployable protocol, in which every method uses the identical observation-fed repair layer. Lower is better; best per column in bold. Full observation and the LP oracle are references, not competitors.

Method	observation ratio ρ			
	0.02	0.05	0.1	0.3
SpaTE	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TEST-style [13]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
zero-fill [10], [9]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
mean-interp	[TODO:]	[TODO:]	[TODO:]	[TODO:]
nbr-fill	[TODO:]	[TODO:]	[TODO:]	[TODO:]
untrained	[TODO:]	[TODO:]	[TODO:]	[TODO:]
full observation ($\rho=1$)			[TODO:]	
LP oracle (lower bound)			[TODO:]	

mask-aware representation matters most. Second, the ordering among baselines follows the information they retain: zero-filling, which conflates “unmeasured” with “zero”, trails the completion-based baselines, and the TEST-style combination of zero-fill plus temporal encoding recovers part of the gap but not all of it, confirming that history alone cannot compensate for a misleading current input. Third, the gap between SpaTE and the full-observation reference narrows as ρ grows ([TODO: numbers]), quantifying how much of the complete-matrix performance sparse operation preserves at each metering budget.

5.3 Learning and Repair Are Complementary

The overall result decomposes into two additive contributions, which we isolate by switching each component off.

Learning contributes on top of repair. Comparing SpaTE with the untrained control in Table 6, both behind the identical deployable repair, isolates what training the sparse-aware allocator adds once repair is already correcting overloads: [TODO: paired wins and median gain per ρ from overall_main.csv]. The gain is largest where the observation is too sparse for the repair estimate to be accurate, which is exactly the regime the learned prior is trained for.

Learning is substantial on its own. With the repair layer switched off entirely, training improves the realized MLU at every observability level tested: 13 of 15 paired cells across $\rho \in [0.02, 0.5]$ (two-sided sign test $p = 0.018$), with a median gain peaking at +28.1% at $\rho = 0.3$ and remaining at +16.5% even at $\rho = 0.02$ (Fig. 4). There is no observability cutoff below which the learned signal collapses: the mask embeddings continue to regularize allocations when only 2% of demands are observed.

Repair contributes what learning cannot. Conversely, the deployable repair corrects the residual overloads that any finite-capacity policy leaves: on the untrained control at $\rho = 0.3$ it cuts the median MLU from 2.833 to 1.740 (Table 7). The two components address different errors, allocation quality versus feasibility residuals, and neither subsumes the other under the deployable protocol: removing either one degrades the end-to-end MLU ([TODO: combined numbers from overall_main.csv]).

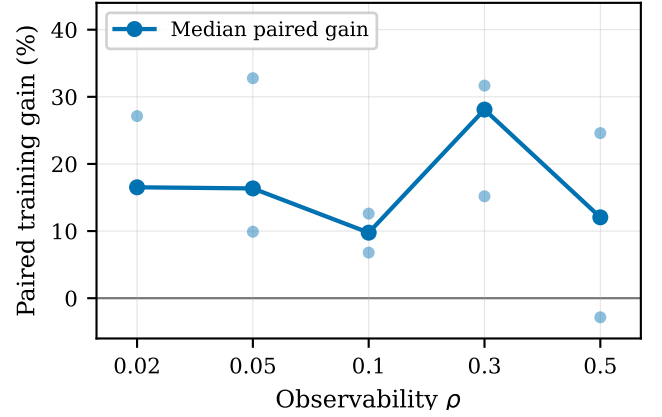


Fig. 4. Training gain without repair as a function of the observation ratio. Learning improves over the untrained control at every ρ tested; the gain is non-monotone but never collapses.

TABLE 7

What repair is worth as a function of the matrix it reads: median MLU of the untrained control at $\rho = 0.3$ (3 seeds), so that only the information given to repair varies. ORACLE is not deployable and shown for reference.

Repair setting	Matrix repair reads	Median MLU
none	n/a	2.833
OBS-ZERO	observation, zeros elsewhere	2.295
OBS-NBR	observation, neighbor fill	1.740
ORACLE	ground truth	1.599

5.4 Designing the Repair Information Source

The deployable repair layer is itself a design contribution: its effectiveness depends critically on *which* traffic estimate its rebalancing sweeps read, a degree of freedom that full-observation systems never had to consider.

The information ladder. Table 7 evaluates the same rebalancing algorithm fed with three different matrices, on the untrained control at $\rho = 0.3$ so that no learning is involved and only the information given to repair varies. The ordering is unambiguous: zero-filled repair recovers only a fraction of the possible improvement (median MLU 2.295 from 2.833), because zeros hide the load that unmeasured traffic places on links and the sweeps misrank exactly the paths that matter. The neighbor estimate closes most of the gap to the non-deployable oracle (1.740 vs. 1.599), confirming that a per-source completion carries enough signal to rank congested paths nearly as well as the ground truth. The answer to “how should repair be designed under sparse observation” is therefore concrete: keep the rebalancing algorithm, but feed it the best observation-derived estimate available; the choice of that estimate is worth more than additional sweeps.

Why the oracle variant is excluded from the main comparison. Beyond being undeployable, oracle-fed repair distorts method comparisons: because it already knows every demand volume, a few sweeps drive very different allocations toward the same equilibrium and flatten the differences between allocators. Table 8 makes this concrete by sweeping the repair budget at $\rho = 0.3$ under oracle repair: a single budget unit collapses the +28.1% training

TABLE 8

Oracle-fed repair flattens method differences: median MLU and paired training gain at $\rho = 0.3$ as the oracle repair budget grows (3 seeds). One budget unit erases the training differential visible at budget zero, which motivates the deployable protocol of Section 5.2.

	oracle repair budget (sweeps of ten)			
	0	1	2	5
Untrained (median MLU)	2.833	1.597	1.599	1.599
Trained (median MLU)	2.184	1.655	1.649	1.649
Training gain	+28.1%	−3.3%	−9.3%	−6.3%
Paired wins	3/3	1/3	1/3	1/3

TABLE 9

Stability under repair: seed spread (max–min) and worst-case MLU of SpaTE vs. zero-fill (5 seeds). Spread: SpaTE wins 5/5; worst case: 4/5.

ρ	Spread		Worst case	
	SpaTE	zero-fill	SpaTE	zero-fill
0.5	0.358	0.456	1.940	1.913
0.3	0.164	0.472	1.761	2.066
0.1	0.398	0.466	1.917	2.072
0.05	0.306	0.432	1.781	2.072
0.02	0.269	1.248	1.694	2.734

gain to −3.3%, and further sweeps change nothing. Under the same oracle repair, the model given the complete matrix is even indistinguishable from sparse models (median MLU 1.692 at $\rho = 1$ vs. 1.587–1.665 at $\rho \leq 0.5$), although more observation can never truly hurt. An evaluation protocol that grants repair the ground truth therefore cannot certify the value of any sparse-input design, which is why our main comparison in Section 5.2 restricts every method to observation-fed repair.

5.5 Stability across Seeds

Beyond the median, a deployable controller cares about the tails: telemetry degradation must not occasionally produce a catastrophic allocation. Table 9 and Fig. 5 compare the seed spread and the worst-case MLU of SpaTE against zero-fill per ρ (5 seeds, repair enabled): SpaTE has the smaller spread at all five observability levels and the better worst case at four of five, with the margin widening as sparsity grows, up to +38.0% at $\rho = 0.02$, where zero-filling’s worst seed degrades to 2.734 while SpaTE stays at 1.694. The learned mask embeddings act as a calibrated prior that protects against unlucky demand-seed combinations, precisely the property a controller needs when the metered subset is small.

5.6 Cross-Scale Generality

To test whether the results depend on constellation scale, we replicate the decisive cells on a one-third-scale instance of the same Shell-1 architecture (528 satellites, 2,112 ISLs, 93 traffic snapshots, demand density $\sim 10\times$ higher per node pair), reusing the weights-shared architecture without any structural change. Table 10 reports the trained/untrained pairing with and without repair at $\rho \in \{0.3, 0.05\}$. The qualitative picture carries over: learning contributes strongly in the repair-free regime (+24.4% at $\rho = 0.05$, 3/3 paired

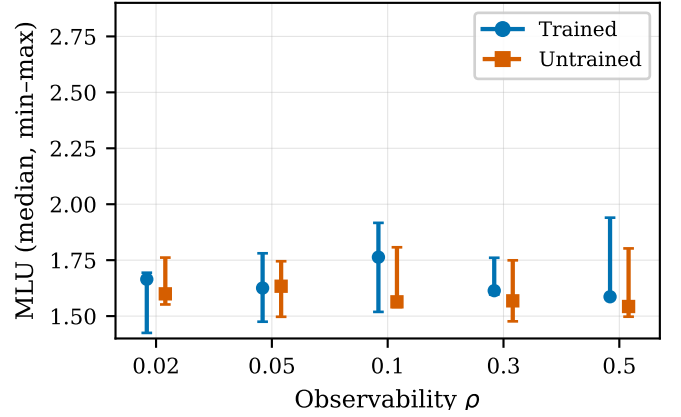


Fig. 5. Per- ρ MLU with repair on (markers show the median, whiskers the min–max over 5 seeds). SpaTE’s whiskers are tighter at every ρ and its worst case is markedly better as sparsity grows.

TABLE 10

Cross-scale check on the 528-satellite instance (median MLU and paired training gain over 3 seeds), with and without the repair layer.

Repair	ρ	Trained	Untrained	Gain	Wins
on	0.3	2.91	2.97	+5.7%	2/3
	0.05	2.86	2.89	−0.6%	1/3
off	0.3	3.69	3.83	−3.2%	1/3
	0.05	3.04	4.02	+24.4%	3/3

wins), and the repair layer delivers a large absolute improvement on top. The observability level at which learning helps most shifts with the heavier per-node load (the repair-free peak moves from $\rho = 0.3$ on the main constellation to $\rho = 0.05$ here), which is expected: what constitutes “enough observation to learn from” depends on traffic density, not on the mechanism itself.

5.7 Inference Latency

On the 1,584-satellite constellation, one full inference pass (demand-set assembly, graph rebuild, gated GNN, temporal encoder, fusion, policy head, and the two-sweep repair) completes in under 0.4 s per snapshot on a single GPU, against a control-loop period of 5 minutes. The repair step is structurally cheap: two sweeps amount to twenty sparse scatter operations over the path-link incidence, so the observation-fed redesign of Section 5.4 costs nothing in latency relative to the inherited implementation. Combined with the $O(D)$ complexity of Section 4.7 and the 4,468-parameter model size, SpaTE comfortably fits the compute envelope of a constellation controller.

5.8 Takeaways

- Under sparse observation with a deployable, observation-fed repair layer, SpaTE delivers the lowest realized MLU across the entire observability range tested, and the advantage grows as observability becomes scarcer.
- Learning and repair are complementary: the learned sparse-aware allocator improves the allocation prior,

the observation-fed repair corrects residual overloads, and removing either degrades end-to-end performance.

- How repair is designed matters as much as whether it is present: feeding the rebalancing a neighbor-based estimate recovers most of the (undeployable) oracle's benefit, whereas zero-filling forfeits half of it. Evaluations that let repair read the ground truth flatten method differences and should be avoided when comparing sparse-input TE systems.

6 CONCLUSION

This paper addressed traffic engineering for LEO satellite constellations under sparse traffic observation, the operating mode that onboard resource limits and telemetry costs impose in practice: a controller that meters only a small fraction of demands must still allocate all traffic within milliseconds. We presented SpaTE, a learning-based TE system designed end to end around the observation actually available. On the allocator side, SpaTE represents unobserved demands with a learnable mask embedding, gates their message passing so that placeholders never contaminate link states, and fuses the history of sparse observations with the spatial graph representation through per-demand cross-attention. On the repair side, SpaTE identifies that the feasibility-repair step inherited from full-observation systems implicitly reads the ground-truth traffic matrix, and replaces that dependence with a neighbor-based estimate computed purely from the observation, showing that the information source handed to repair, rather than the repair budget, is what determines its effectiveness under sparsity.

On a 1,584-satellite Starlink constellation with population-driven traffic, SpaTE achieves the lowest realized MLU among all sparse adaptations of existing systems at every observation ratio from 2% to 50% under the identical deployable repair layer ([**TODO: headline numbers from overall_main.csv**]). The two components are complementary: learning improves the allocation prior at every observability level, repair corrects residual overloads, and the neighbor-fed estimate recovers most of the benefit of the undeployable oracle repair. SpaTE additionally offers markedly lower seed variance and better worst-case MLU than zero-filling, transfers to a one-third-scale constellation without retraining, and infers in under 0.4 s per snapshot with 4,468 parameters, comfortably within a constellation controller's compute envelope.

Three directions remain open. First, replacing the heuristic rebalancing with a differentiable feasibility layer trained jointly with the allocator may let the two components share, rather than split, the observation. Second, the stability advantage suggests a dedicated training objective that explicitly optimizes worst-case MLU rather than the mean. Third, measurement scheduling, deciding *which* demands to observe next, is a natural extension: once both the allocator and the repair estimate are driven by the observation, each additional measurement improves them jointly, and the observation budget itself becomes a control variable.

REFERENCES

- [1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*, 2018.
- [2] D. Bhattacharjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.
- [3] G. Giuliari, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, "Internet backbones in space," *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, 2020.
- [4] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the "internet from space" with hypatia," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.
- [5] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at starlink performance," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.
- [6] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013, pp. 3–14.
- [7] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.
- [8] C.-Y. Hong, S. Mandal, M. Al-Fares, M. Zhu, R. Alimi, K. N. Bollineni, C. Bhagat, S. Jain, J. Kaimal, S. Liang, K. Mendelev, S. Padgett, F. Rabe, S. Ray, M. Tewari, M. Tierney, M. Zahn, J. Zolla, J. Ong, and A. Vahdat, "B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined WAN," in *Proceedings of the ACM SIGCOMM 2018 Conference*, 2018, pp. 74–87.
- [9] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, "SaTE: Low-latency traffic engineering for satellite networks," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 896–916.
- [10] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of WAN traffic engineering," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.
- [11] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, "DOTE: Rethinking (predictive) WAN traffic engineering," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1557–1581.
- [12] A. A. AlQiam, Y. Yao, Z. Wang, S. S. Ahuja, Y. Zhang, S. G. Rao, B. Ribeiro, and M. Tawarmalani, "Transferable neural WAN TE for changing topologies," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 86–102.
- [13] Y. Guo, Z. Huang, M. Ding, and H. Luo, "An end-to-end learning approach for traffic engineering with sparse traffic measurements," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 5448–5463, 2026.
- [14] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.
- [15] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM 2000*, 2000.
- [16] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, "COPE: Traffic engineering in dynamic networks," in *Proceedings of the ACM SIGCOMM 2006 Conference*, 2006.
- [17] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C. L. Lim, and R. Soulé, "Semi-oblivious traffic engineering: The road not taken," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 157–170.
- [18] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, and P. Bailis, "Contracting wide-area network topologies to solve flow problems quickly," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 175–200.
- [19] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, "Solving large-scale granular resource allocation problems efficiently with POP," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 521–537.

- [20] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, 2017.
- [21] X. Liu, S. Zhao, Y. Cui, and X. Wang, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 117–135.
- [22] Y. Fan, J. Liu, P. Xie, and L. Liu, "Aether: Toward generalized traffic engineering with elastic multi-agent graph transformers," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
- [23] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "Path-based graph neural network for robust and resilient routing in distributed traffic engineering," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 2, pp. 422–436, 2025.
- [24] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 661–678.
- [25] Y. Lu, F. Sun, and Y. Zhao, "Virtual topology for LEO satellite networks based on earth-fixed footprint mode," *IEEE Communications Letters*, vol. 17, no. 2, pp. 357–360, 2013.
- [26] M. Hu, M. Xiao, W. Xu, T. Deng, Y. Dong, and K. Peng, "Traffic engineering for software-defined LEO constellations," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5090–5103, 2022.
- [27] L. Wei, T. T. Le, Y. Ji, M. Wang, Y. Liu, Y. Wang, and J. C. S. Lui, "Towards efficient traffic engineering via distributed optimization in large-scale LEO constellation," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 353–358.
- [28] X. Liu, Z. Zhang, X. Qin, H. Zhou, and L. Zhao, "FlexSATE: Flexible and distributed traffic engineering with supervised learning in ultra-dense low-earth-orbit satellite networks," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 4466–4471.
- [29] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, C. Ying, T. Liu, S. Yue, and H. Jin, "Scalable traffic allocation in dynamic networks via end-to-end imitation learning," *IEEE/ACM Transactions on Networking*, vol. 34, 2026.
- [30] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, T. Liu, and H. Jin, "Learning to accelerate traffic allocation over large-scale networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.
- [31] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American Statistical Association*, vol. 91, no. 433, 1996.
- [32] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proceedings of ACM SIGMETRICS 2003*, 2003.
- [33] Y. Zhang, M. Roughan, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices," in *Proceedings of the ACM SIGCOMM 2009 Conference*, 2009.
- [34] C. Huo, D. Jin, Y. Li, D. He, Y.-B. Yang, and L. Wu, "T2-GNN: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [35] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [37] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [38] G. Giulari, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 317–331.